

only be started when the app was in a state where it was considered okay to do so, such as any time the app itself was already in the foreground for another reason, or it was executing in response to something that could be related to a user intent (such as responding to the aforementioned media button event).

This leaves us at the state of background execution in Android, circa 2021. Android will, however, continue to evolve; not only to continue to optimize the battery life it can provide, but also as it has to address the changing behavior of its application ecosystem and expectations of its users.

10.9 SUMMARY

In this chapter we have looked at two examples in detail: Linux and Android, which built on top on Linux. Linux has been around now for a bit over 30 years and has grown from a hobby project by one person who wanted a production version of MINIX to a large and powerful system that powers most of the Internet. It is also the most successful open-source project in history.

We started with a brief overview of the user interface and the shell, with some examples of what you can do on the command line. Then we took a brief look at some of the standard UNIX programs that are available in Linux. Next we saw how Linux is structured in layers.

After that we moved onto the core of the Linux material, how it works inside. This included processes and threads, memory management, input/output, the file system, and, of course, security. For each of these we showed some of the system calls available and how they are implemented. Then we moved on to Android, which is layered on top of Linux. Linux itself is mostly used on desktops, notebooks, and servers but Android is aimed at mobile devices such as smartphones and tablets. This changes its goals and requirements considerably. For example, how long it takes to start a program is of only minor interest on notebooks, but it is crucial on mobile devices. Notebook users really don't mind if Word takes 3 seconds to start, but smartphone users would go crazy if hitting the app to make a phone call took 3 seconds to boot. This simple difference in goals has vast implications for the respective designs.

Another enormous difference between Linux and Android is that while Linux tries to avoid wasting energy, Android goes to great lengths to prevent draining the battery to fast. A smartphone whose battery lasted only 4 hours would not be a big hit.

After going over the design goals and history of Android, we took a look at the system architecture. Then we studied the innards in detail, including ART, binder IPC, how apps work, intents, and the Android process model. Next come the ever-important security, which has evolved over the years in Android as it has become more important. Finally we looked at background execution, which is quite different from how it is done on desktops and notebooks.

PROBLEMS

1. The first version of UNIX was written in assembly code. Explain how writing UNIX in C made it easier to port it to new machines.
2. What is a portable C compiler? How does it simplify portability of UNIX?
3. The POSIX interface defines a set of library procedures. Explain why POSIX standardizes library procedures instead of the system-call interface.
4. Linux depends on gcc compiler to be ported to new architectures. Describe one advantage and one disadvantage of this dependency.
5. When the kernel catches a system call, how does it know which system call it is supposed to carry out?
6. What is the difference, if any between these two Linux command lines? Think about all possible cases.

```
cat f1 f2 f3 | grep "day" | head -500
```

```
cat f1 f2 f3 | grep "day" >tmp; head -500 tmp; rm tmp
```

7. What does the following Linux shell pipeline do?

```
grep rt xyz | wc -l
```

8. When the Linux shell starts up a process, it puts copies of its environment variables, such as *HOME*, on the process' stack, so the process can find out what its home directory is. If this process should later fork, will the child automatically get these variables, too?
9. About how long does it take a traditional UNIX system to fork off a child process under the following conditions: text size = 100 KB, data size = 20 KB, stack size = 10 KB, task structure = 1 KB, user structure = 5 KB. The kernel trap and return takes 1 msec, and the machine can copy one 32-bit word every 50 nsec. Text segments are shared, but data and stack segments are not.
10. As multimegabyte programs became more common, the time spent executing the fork system call and copying the data and stack segments of the calling process grew proportionally. When fork is executed in Linux, the parent's address space is not copied, as traditional fork semantics would dictate. How does Linux prevent the child from doing something that would completely change the fork semantics?
11. Why are negative arguments to nice reserved exclusively for the superuser?
12. A non-real-time Linux process has priority levels from 100 to 139. What is the default static priority and how is the nice value used to change this?
13. Does it make sense to take away a process' memory when it enters zombie state? Why or why not?

14. To what hardware concept is a signal closely related? Give two examples of how signals are used.
15. Why do you think the designers of Linux made it impossible for a process to send a signal to another process that is not in its process group?
16. There are a number of daemons running on most UNIX systems including Linux. Identify five daemons and provide a short description of each one. (*Hint*: Think about networking.)
17. When a new process is forked off, it must be assigned a unique integer as its PID. Is it sufficient to have a counter in the kernel that is incremented on each process creation, with the counter used as the new PID? Discuss your answer.
18. In every process' entry in the task structure, the PID of the parent is stored. Why?
19. The copy-on-write mechanism is used as an optimization in the fork system call, so that a copy of a page is created only when one of the processes (parent or child) tries to write on the page. Suppose a process p_1 forks processes p_2 and p_3 in quick succession. Explain how a page sharing may be handled in this case.
20. Two tasks A and B need to perform the same amount of work. However, task A has higher priority, and needs to be given more CPU time. Explain how will this be achieved in each of the Linux schedulers described in this chapter, the O(1) and the CFS scheduler.
21. Some UNIX systems are tickless, meaning they do not have periodic clock interrupts. Why is this done? Also, does ticklessness make sense on a computer (such as an embedded system) running only one process?
22. When booting Linux (or most other operating systems for that matter), the bootstrap loader in sector 0 of the disk first loads a boot program which then loads the operating system. Why is this extra step necessary? Surely it would be simpler to have the bootstrap loader in sector 0 just load the operating system directly.
23. A certain editor has 100 KB of program text, 30 KB of initialized data, and 50 KB of BSS. The initial stack is 10 KB. Suppose that three copies of this editor are started simultaneously. How much physical memory is needed (a) if shared text is used, and (b) if it is not?
24. Why are open-file-descriptor tables necessary in Linux?
25. In Linux, the data and stack segments are paged and swapped to a scratch copy kept on a special paging disk or partition, but the text segment uses the executable binary file instead. Why?
26. A DAX file system does not use a page cache. When is such a file system appropriate? Would you use a DAX file system with a hard disk? Why or why not?
27. Describe a way to use mmap and signals to construct an interprocess-communication mechanism.

28. A file is mapped in using the following `mmap` system call:

```
mmap(65536, 32768, READ, FLAGS, fd, 0)
```

Pages are 8 KB. Which byte in the file is accessed by reading a byte at memory address 72,000?

29. After the system call of the previous problem has been executed, the call

```
munmap(65536, 8192)
```

is carried out. Does it succeed? If so, which bytes of the file remain mapped? If not, why does it fail?

30. Can a page fault ever lead to the faulting process being terminated? If so, give an example. If not, why not?
31. Is it possible that with the buddy system of memory management it ever occurs that two adjacent blocks of free memory of the same size coexist without being merged into one block? If so, explain how. If not, show that it is impossible.
32. It is stated in the text that a paging partition will perform better than a paging file. Why is this so?
33. Give two examples of the advantages of relative path names over absolute ones.
34. The following locking calls are made by a collection of processes. For each call, tell what happens. If a process fails to get a lock, it blocks.
- (a) *A* wants a shared lock on bytes 0 through 10.
 - (b) *B* wants an exclusive lock on bytes 20 through 30.
 - (c) *C* wants a shared lock on bytes 8 through 40.
 - (d) *A* wants a shared lock on bytes 25 through 35.
 - (e) *B* wants an exclusive lock on byte 8.
35. Consider the locked file of Fig. 10-26(c). Suppose that a process tries to lock bytes 10 and 11 and blocks. Then, before *C* releases its lock, yet another process tries to lock bytes 10 and 11, and also blocks. What kinds of problems are introduced into the semantics by this situation? Propose and defend two solutions.
36. Explain under what situations a process may request a shared lock or an exclusive lock. What problem may a process requesting an exclusive lock suffer from?
37. Suppose that an `lseek` system call seeks to a negative offset in a file. Given two possible ways of dealing with it.
38. If a Linux file has protection mode 755 (octal), what can the owner, the owner's group, and everyone else do to the file?
39. Some tape drives have numbered blocks and the ability to overwrite a particular block in place without disturbing the blocks in front of or behind it. Could such a device hold a mounted Linux file system?

40. In Fig. 10-24, both Aron and Nathan have access to the file *x* in their respective directories after linking. Is this access completely symmetrical in the sense that anything one of them can do with it the other one can, too?
41. As we have seen, absolute path names are looked up starting at the root directory and relative path names are looked up starting at the working directory. Suggest an efficient way to implement both kinds of searches.
42. When the file */usr/ast/work/f* is opened, several disk accesses are needed to read i-node and directory blocks. Calculate the number of disk accesses required under the assumption that the i-node for the root directory is always in memory, and all directories are one block long.
43. A Linux i-node has 12 disk addresses for data blocks, as well as the addresses of single, double, and triple indirect blocks. If each of these holds 256 disk addresses, what is the size of the largest file that can be handled, assuming that a disk block is 1 KB?
44. When an i-node is read in from the disk during the process of opening a file, it is put into an i-node table in memory. This table has some fields that are not present on the disk. One of them is a counter that keeps track of the number of times the i-node has been opened. Why is this field needed?
45. On multi-CPU platforms, Linux maintains a *runqueue* for each CPU. Is this a good idea? Explain your answer?
46. The concept of loadable modules is useful in that new device drivers may be loaded in the kernel while the system is running. Provide two disadvantages of this concept.
47. The kernel worker threads can be awakened periodically to write back to disk very old pages—older than 30 sec. Why is this necessary?
48. After a system crash and reboot, a recovery program is usually run. Suppose this program discovers that the link count in a disk i-node is 2, but only one directory entry references the i-node. Can it fix the problem, and if so, how?
49. Based on the information presented in this chapter, if a Linux ext2 file system were to be put on a 1.44-MB floppy disk, what is the maximum amount of user file data that could be stored on the disk? Assume that disk blocks are 1 KB.
50. In view of all the trouble that students can cause if they get to be superuser, why does this concept exist in the first place?
51. A professor shares files with his students by placing them in a publicly accessible directory on the Computer Science department's Linux system. One day he realizes that a file placed there the previous day was left world-writable. He changes the permissions and verifies that the file is identical to his master copy. The next day he finds that the file has been changed. How could this have happened and how could it have been prevented?
52. Linux has a system call *fsuid*. Unlike *setuid*, which grants the user all the rights of the effective id associated with a running program *fsuid* grants the user who is running the program special rights only with respect to access to files. Why is this feature useful?

53. On a Linux system, go to `/proc/####` directory, where `####` is a decimal number corresponding to a process currently running in the system. Answer the following along with an explanation:
- (a) What is the size of most of the files in this directory?
 - (b) What are the time and date settings of most of the files?
 - (c) What type of access right is provided to the users for accessing the files?
54. If you are writing an Android activity to display a Web page in a browser, how would you implement its activity-state saving to minimize the amount of saved state without losing anything important?
55. If you are writing networking code on Android that uses a socket to download a file, what should you consider doing that is different than on a standard Linux system?
56. If you are designing something like Android's *zygote* process for a system that will have multiple threads running in each process forked from it, would you prefer to start those threads in *zygote* or after the fork?
57. Imagine you use Android's Binder IPC to send an object to another process. You later receive an object from a call into your process, and find that what you have received is the same object as previously sent. What can you assume or not assume about the caller in your process?
58. Consider an Android system that, immediately after starting, follows these steps:
1. The home (or launcher) application is started.
 2. The email application starts syncing its mailbox in the background.
 3. The user launches a camera application.
 4. The user launches a Web browser application.

The Web page the user is now viewing in the browser application requires increasingly more RAM, until it needs everything it can get. What happens?

59. Consider the following Binder IPC scenario. Process *P1* has a Binder object implementing interface *I1*, Process *P2* has a Binder object implementing interface *I2*, Process *P3* has a Binder object implementing interface *I3*. Process *P1* creates a new Binder object with interface *Ie*; *P1* calls *I2* to send *Ie* to *P2*, then *P2* calls *I3* to send that *Ie* to *P3*, then *P3* calls *I1* to send that *Ie* to *P1*. *P1* now takes the *Ie* it received from *P3* and calls a method on it. What happens and why?
60. We have the following user journey through the Android system. Each application has one process associated with it.
1. Launch a “media player” application, and start playing music. The media player starts a foreground service to play the music.
 2. The “media player” while playing music uses a content provider in another “audio server” app to retrieve the audio data it is playing.
 3. Now return home, and start a “messaging” app.
 4. In the “messaging” app, send a message to a friend, attaching an audio file. The messaging app is now using the content provider in the “audio server” app to retrieve the audio file.

5. While this is happening, a “email” app runs in the background to retrieve new messages from its server.

At this point with what we know, what is the importance category of the “media player,” “audio server,” “messaging,” and “email” processes?

61. You have been told that Android has too many runtime permission prompts being shown to users, and you need to get rid of one of them. The current runtime prompts are (in the text shown to the user) “Contacts (access your contacts),” “Calendar (access your calendar),” “SMS (send and view SMS messages),” “Storage (access photos, media, and files),” “Location (access device’s location),” “Phone (make and manage phone calls),” “Microphone (record audio),” “Camera (take pictures and record video),” and “Body sensors (access sensor data about your vital signs).” Which of these would you select to try to remove, and why?
62. You are starting to see a problem where users are doing many more explicit upload/download operations (such as sending large videos and recordings and downloading them), which apps correctly implement as foreground services. However, on devices that are more limited in RAM, these are conflicting with other foreground services like music playback, causing situations where the user’s music is killed instead of uploads/downloads that would be a better choice. How might you solve this?
63. Write a minimal shell that allows simple commands to be started. It should also allow them to be started in the background.
64. Write a dumb terminal program to connect two Linux computers via the serial ports. Use the POSIX terminal management calls to configure the ports.
65. Write a client-server application which, on request, transfers a large file via sockets. Reimplement the same application using shared memory. Which version do you expect to perform better? Why? Conduct performance measurements with the code you have written and using different file sizes. What are your observations? What do you think happens inside the Linux kernel which results in this behavior?
66. Implement a basic user-level threads library to run on top of Linux. The library API should contain function calls like `mythreads_init`, `mythreads_create`, `mythreads_join`, `mythreads_exit`, `mythreads_yield`, `mythreads_self`, and perhaps a few others. Next, implement these synchronization variables to enable safe concurrent operations: `mythreads_mutex_init`, `mythreads_mutex_lock`, `mythreads_mutex_unlock`. Before starting, clearly define the API and specify the semantics of each of the calls. Next implement the user-level library with a simple, round-robin preemptive scheduler. You will also need to write one or more multithreaded applications, which use your library, in order to test it. Finally, replace the simple scheduling mechanism with another one which behaves like the Linux 2.6 O(1) scheduler described in this chapter. Compare the performance your application(s) receive when using each of the schedulers.
67. Write a shell script that displays some important system information such as what processes you are running, your home directory and current directory, processor type, current CPU utilization, etc.

68. Using assembly language and BIOS calls, write a program that boots itself from a USB drive on an x86 computer. The program should use BIOS calls to read the keyboard and echo the characters typed, just to demonstrate that it is running.